

Classification of Cancer outcome using Genetic and Clinical data

Ewen Expuesto and Candice Aubertin

Data Analysis

```
load("mrr_bio.Rdata")
library(glmnet)
```

```
## Loading required package: Matrix
## Loaded glmnet 4.1-10
```

Genes

We will first try to do a LASSO regression only with the genes data to predict the vital status of the patients.

Data preparation and encoding for vital_status:

```
# Outcome and keep only rows without missing values
y <- factor(clinical_data$vital_status)
```

```
## Loading required namespace: S4Vectors
```

```
keep <- complete.cases(y, GeneX)
if (any(!keep)) cat("Removing", sum(!keep), "rows with missing values before LASSO\n")
```

```
## Removing 1 rows with missing values before LASSO
```

```
y <- y[keep]
X <- as.matrix(GeneX[keep, , drop = FALSE])
y <- droplevels(y)
```

```
# Encoding vital_status as binary (1 = Dead, 0 = Alive)
pos_label <- if ("Dead" %in% levels(y)) "Dead" else levels(y)[1]
neg_label <- setdiff(levels(y), pos_label)
if (length(neg_label) == 0) neg_label <- pos_label
y_bin <- as.numeric(y == pos_label)
```

Fit LASSO models with cross validation and plot coefficient paths:

```
lasso_genes <- glmnet(X, y_bin, alpha = 1, family = "binomial")
matplot(
  -log(lasso_genes$lambda),
  t(as.matrix(lasso_genes$beta)),
  type = "l",
  lty = 1,
```

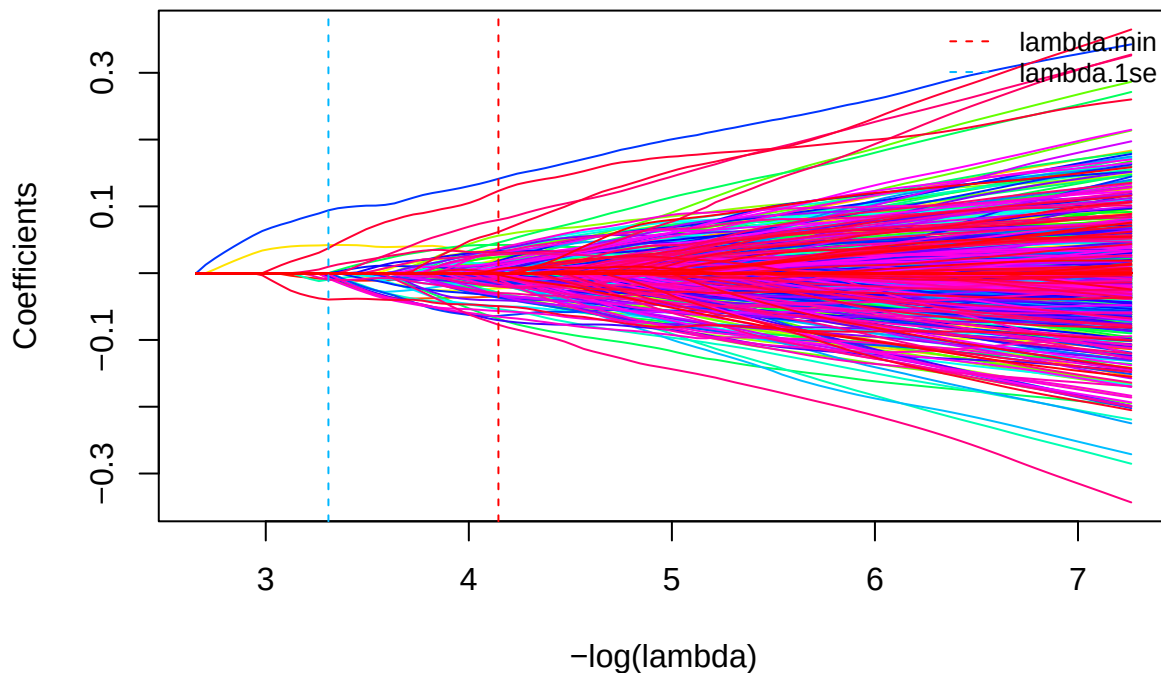
```

col = rainbow(ncol(X)),
xlab = "-log(lambda)",
ylab = "Coefficients",
main = "LASSO Coefficient Paths"
)

cv_lasso <- cv.glmnet(X, y_bin, alpha = 1, family = "binomial", grouped = FALSE, nfolds = 10)
abline(v = -log(cv_lasso$lambda.min), col = "red", lty = 2)
abline(v = -log(cv_lasso$lambda.1se), col = "#00b7ff", lty = 2)
legend("topright",
      legend = c("lambda.min", "lambda.1se"),
      col = c("red", "#00b7ff"),
      lty = 2,
      bty = "n",
      cex = 0.8)

```

LASSO Coefficient Paths

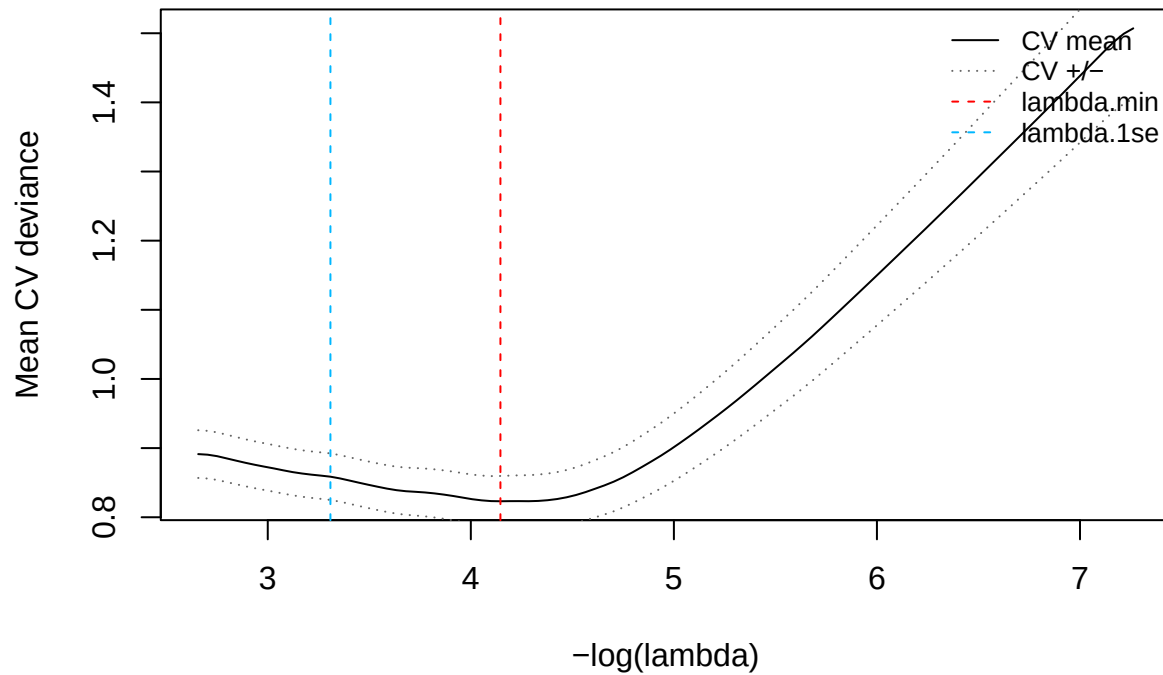


```

# CV curve for the gene LASSO (custom plot to avoid top labels)
lambda_nlog <- -log(cv_lasso$lambda)
plot(lambda_nlog, cv_lasso$cvm, type = "l",
     xlab = "-log(lambda)", ylab = "Mean CV deviance",
     main = "Gene LASSO CV Curve")
lines(lambda_nlog, cv_lasso$cvup, lty = 3, col = "gray40")
lines(lambda_nlog, cv_lasso$cvlo, lty = 3, col = "gray40")
abline(v = -log(cv_lasso$lambda.min), col = "red", lty = 2)
abline(v = -log(cv_lasso$lambda.1se), col = "#00b7ff", lty = 2)
legend("topright", bty = "n", cex = 0.8,
      legend = c("CV mean", "CV +/-", "lambda.min", "lambda.1se"),
      lty = c(1, 3, 2, 2), col = c("black", "gray40", "red", "#00b7ff"))

```

Gene LASSO CV Curve



Print lambda values:

```
cat("lambda.min:", cv_lasso$lambda.min, "\n")
```

```
## lambda.min: 0.01583061
```

```
cat("lambda.1se:", cv_lasso$lambda.1se, "\n")
```

```
## lambda.1se: 0.03657076
```

Predictions at lambda.min and lambda.1se:

```
p_min <- as.numeric(predict(cv_lasso, newx = X, s = "lambda.min", type = "response"))
```

```
p_1se <- as.numeric(predict(cv_lasso, newx = X, s = "lambda.1se", type = "response"))
```

```
cat("Observed class counts (0=", neg_label, ",1=", pos_label, "):\n", sep = "")
```

```
## Observed class counts (0=Alive,1=Dead):
```

```
print(table(y_bin))
```

```
## y_bin
```

```
##    0    1
```

```
## 1029 201
```

```
cat("# positives (p>0.5) at lambda.min:", sum(p_min > 0.5), "of", length(p_min), "\n")
```

```
## # positives (p>0.5) at lambda.min: 34 of 1230
```

```
cat("# positives (p>0.5) at lambda.1se:", sum(p_1se > 0.5), "of", length(p_1se), "\n")
```

```
## # positives (p>0.5) at lambda.1se: 0 of 1230
```

```
cat("Summary probs (lambda.min):\n")
```

```

## Summary probs (lambda.min):
print(summary(p_min))

##      Min.   1st Qu.   Median     Mean 3rd Qu.     Max.
## 0.009085 0.077635 0.127532 0.163415 0.205897 0.795815
cat("Summary probs (lambda.1se):\n")

## Summary probs (lambda.1se):
print(summary(p_1se))

##      Min.   1st Qu.   Median     Mean 3rd Qu.     Max.
## 0.08118 0.13379 0.15328 0.16341 0.18110 0.37007

# Map numeric predictions/observations to labels for readability
obs_label <- ifelse(y_bin == 1, pos_label, neg_label)
pred_label_min <- ifelse(p_min > 0.5, pos_label, neg_label)
pred_label_1se <- ifelse(p_1se > 0.5, pos_label, neg_label)
label_order <- unique(c(neg_label, pos_label))

cm_min <- table(
  Predicted = factor(pred_label_min, levels = label_order),
  Observed = factor(obs_label, levels = label_order)
)
cm_1se <- table(
  Predicted = factor(pred_label_1se, levels = label_order),
  Observed = factor(obs_label, levels = label_order)
)

acc_min <- sum(diag(cm_min)) / sum(cm_min)
acc_1se <- sum(diag(cm_1se)) / sum(cm_1se)

cat(sprintf("Acc (lambda.min): %.3f\n", acc_min))

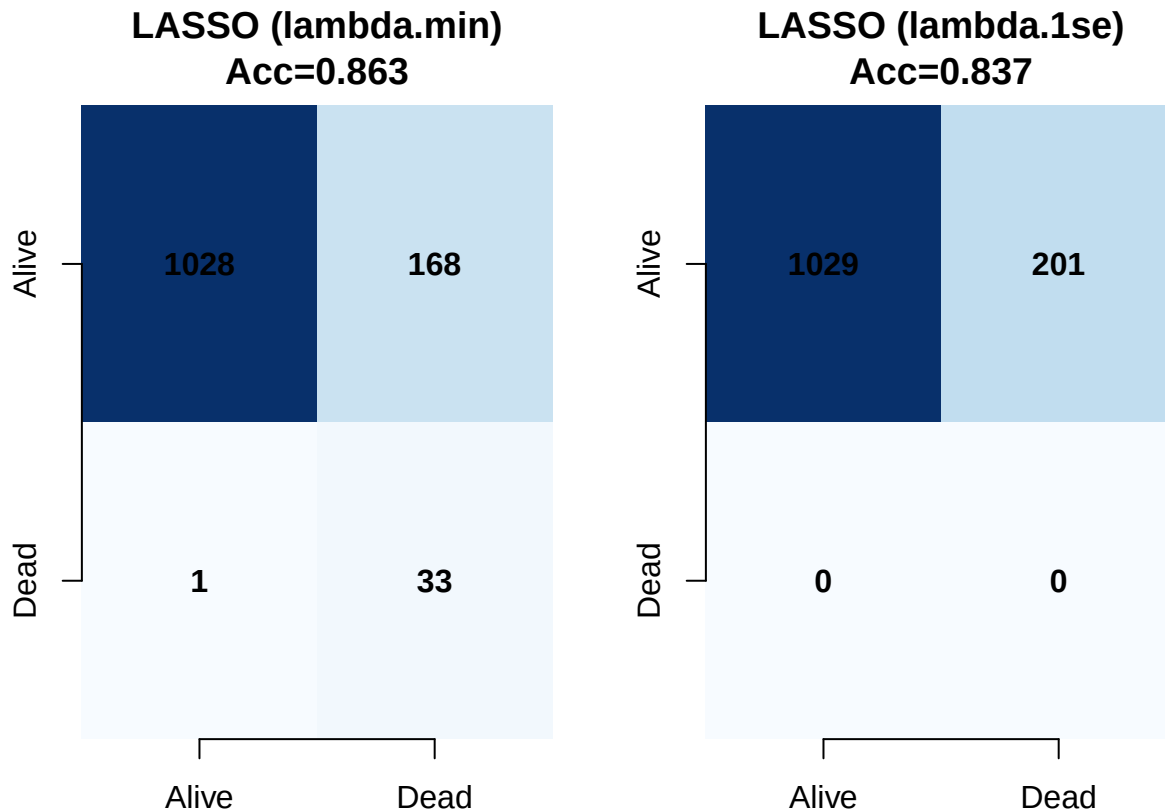
## Acc (lambda.min): 0.863
cat(sprintf("Acc (lambda.1se): %.3f\n", acc_1se))

## Acc (lambda.1se): 0.837

# Plot side-by-side confusion matrices
par(mfrow = c(1,2), mar = c(3,3,3,1))
plot_cm <- function(cm, title, palette = c("#f7fbff", "#6baed6", "#08306b")) {
  m <- as.matrix(cm)
  image(t(m[nrow(m):1, , drop = FALSE]), axes = FALSE,
    col = colorRampPalette(palette)(64), main = title)
  axis(1, at = seq(0,1,length.out = ncol(m)), labels = colnames(m))
  axis(2, at = seq(0,1,length.out = nrow(m)), labels = rev(rownames(m)))
  text(rep(seq(0,1,length.out = ncol(m)), each = nrow(m)),
    rep(seq(0,1,length.out = nrow(m)), times = ncol(m)),
    labels = as.vector(m[rev(seq_len(nrow(m))), ]), col = "black", font = 2)
}

plot_cm(cm_min, paste0("LASSO (lambda.min)\nAcc=", round(acc_min,3)))
plot_cm(cm_1se, paste0("LASSO (lambda.1se)\nAcc=", round(acc_1se,3)))

```



```
# Most important genes for LASSO (lambda.min)
coef_lasso_min <- coef(cv_lasso, s = "lambda.min")
coef_lasso_min_vec <- as.numeric(coef_lasso_min)
names(coef_lasso_min_vec) <- rownames(coef_lasso_min)
important_vars_lasso_min <- sort(abs(coef_lasso_min_vec), decreasing = TRUE)
# Exclude intercept from the ranking
important_vars_lasso_min <- important_vars_lasso_min[names(important_vars_lasso_min) != "(Intercept)"]
head(important_vars_lasso_min, 13)

## LINC01235      PSD2      SLC6A3      SFTPA1      GPS2P1      TNN AL031432.2
## 0.14126189 0.12313383 0.07971687 0.07724159 0.07045672 0.06833976 0.06516908
## SEMA3B AC017048.3 GSTT2B      MPPED1      RPS4Y1      SNORD104
## 0.06405589 0.06164001 0.05673017 0.05621008 0.05383864 0.04939804
```

Clinical data

Data cleaning

Here is each type corresponding to each data point as well as the frequencies of each occurrence.

After analysis we can remove redundant data like `age_at_diagnosis`, `days_to_birth` which are like the age in `age_at_index` and `bcr_patient_barcode` which is like the index of the line, and also `disease_type` and `primary_site` because they contain only one one observation which is the same fall all patients.

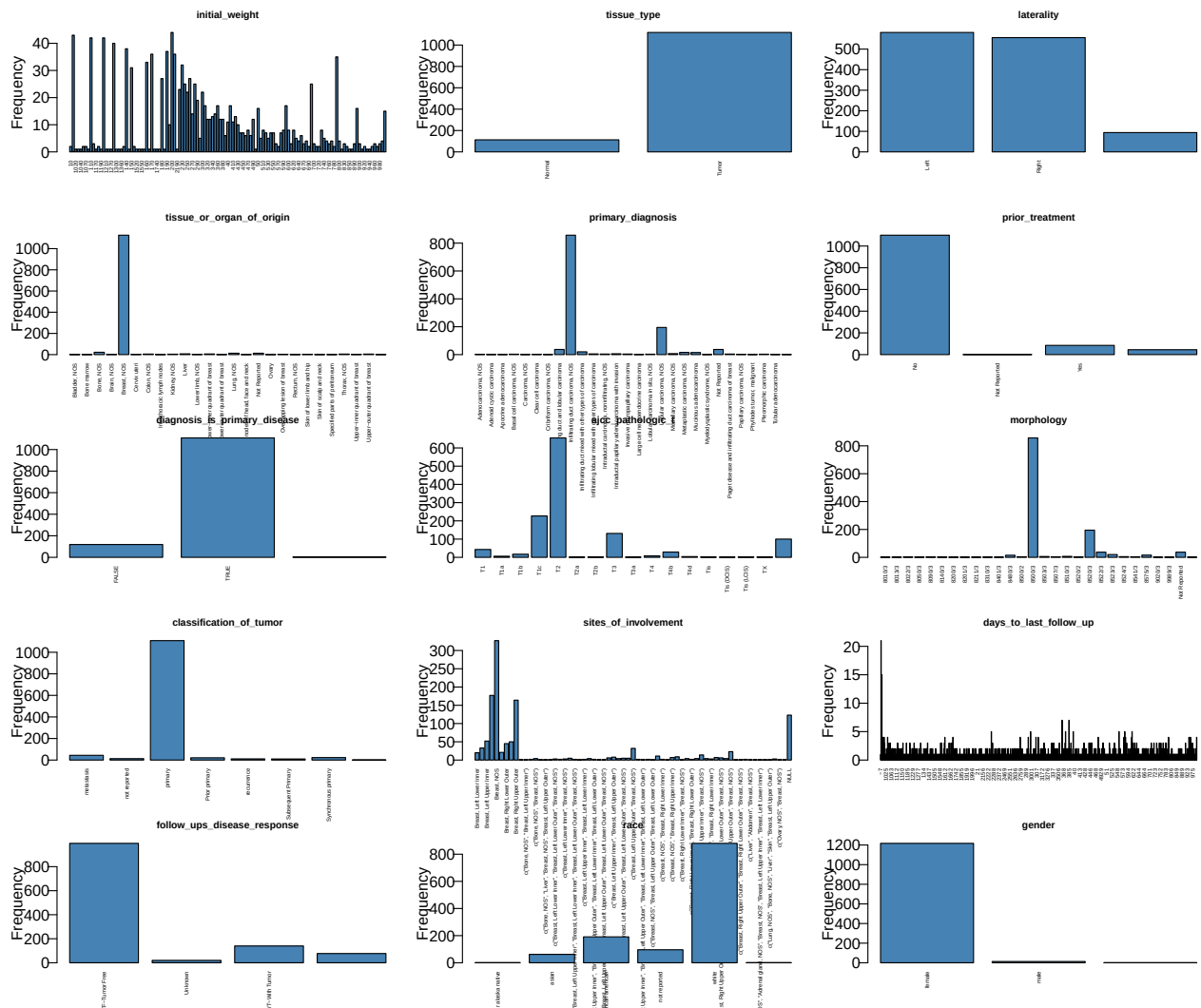
```
redundant_cols <- c("age_at_diagnosis", "days_to_birth", "bcr_patient_barcode",
                    "disease_type", "primary_site")
clinical_data <- clinical_data[, setdiff(names(clinical_data), redundant_cols)]
```

```
cat("Removed columns:", paste(redundant_cols, collapse = ", "), "\n")
```

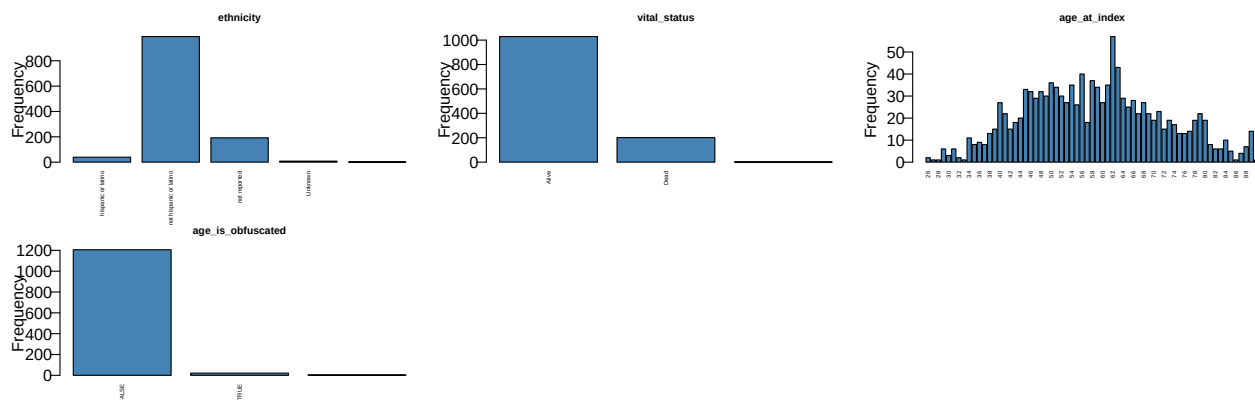
```
## Removed columns: age_at_diagnosis, days_to_birth, bcr_patient_barcode, disease_type, primary_site
```

```
par(mfrow = c(5, 3), mar = c(2.5, 2.5, 2, 1), mgp = c(1.7, 0.4, 0), cex = 0.9)
```

```
for (col in names(clinical_data)) {
  tbl <- table(as.character(clinical_data[[col]]), useNA = "ifany")
  barplot(tbl,
    main = col,
    ylab = "Frequency",
    col = "steelblue",
    las = 2,
    cex.names = 0.4,
    cex.main = 0.65)
}
```



```
par(mfrow = c(1, 1))
```



Encoding categorical variables

Label encoding of the `clinical_data` categorical variables, with NA also as a category. Label encoding (bijection) for non-integer clinical columns :

```
label_encoded_clinical <- clinical_data
label_mappings <- list()

for (col in names(label_encoded_clinical)) {
  column_values <- label_encoded_clinical[[col]]
  if (!is.numeric(column_values)) {
    if (!is.atomic(column_values)) {
      column_values <- vapply(column_values, function(entry) {
        if (is.null(entry) || (length(entry) == 1 && is.na(entry))) {
          return(NA_character_)
        }
        paste0(as.character(entry), collapse = "|")
      }, character(1))
    }
    factor_values <- factor(column_values, exclude = NULL)
    label_encoded_clinical[[col]] <- as.integer(factor_values)
    label_mappings[[col]] <- levels(factor_values)
  }
}

label_encoded_clinical[] <- lapply(label_encoded_clinical, function(col) {
  if (is.numeric(col)) {
    col[is.na(col)] <- -1 # NA placeholder for numeric columns
  } else {
    f <- factor(col, exclude = NULL)
    col <- as.integer(f)
  }
  col
})

cat("Label-encoded clinical data dimensions:",
    paste(dim(label_encoded_clinical), collapse = " x "), "\n")
```

```
## Label-encoded clinical data dimensions: 1231 x 19
```

```

if (length(label_mappings) > 0) {
  cat("Bijections:\n")
  for (col in names(label_mappings)) {
    cat(col, "=>",
        paste(seq_along(label_mappings[[col]]),
              label_mappings[[col]],
              sep = ":", collapse = ", "),
        "\n")
  }
}

## Bijections:
## tissue_type => 1:Normal, 2:Tumor
## laterality => 1:Left, 2:Right, 3:NA
## tissue_or_organ_of_origin => 1:Bladder, NOS, 2:Bone marrow, 3:Bone, NOS, 4:Brain, NOS, 5:Breast, NOS
## primary_diagnosis => 1:Adenocarcinoma, NOS, 2:Adenoid cystic carcinoma, 3:Apocrine adenocarcinoma, 4
## prior_treatment => 1:No, 2:Not Reported, 3:Yes, 4:NA
## diagnosis_is_primary_disease => 1:FALSE, 2:TRUE, 3:NA
## ajcc_pathologic_t => 1:T1, 2:T1a, 3:T1b, 4:T1c, 5:T2, 6:T2a, 7:T2b, 8:T3, 9:T3a, 10:T4, 11:T4b, 12:T
## morphology => 1:8010/3, 2:8013/3, 3:8022/3, 4:8050/3, 5:8090/3, 6:8140/3, 7:8200/3, 8:8201/3, 9:8211
## classification_of_tumor => 1:metastasis, 2:not reported, 3:primary, 4:Prior primary, 5:recurrence, 6
## sites_of_involvement => 1:Bone, NOS|Breast, Left Lower Outer|Breast, NOS, 2:Bone, NOS|Breast, Left U
## follow_ups_disease_response => 1:TF-Tumor Free, 2:Unknown, 3:WT-With Tumor, 4:NA
## race => 1:american indian or alaska native, 2:asian, 3:black or african american, 4:not reported, 5:
## gender => 1:female, 2:male, 3:NA
## ethnicity => 1:hispanic or latino, 2:not hispanic or latino, 3:not reported, 4:Unknown, 5:NA
## vital_status => 1:Alive, 2:Dead, 3:NA
## age_is_obfuscated => 1:FALSE, 2:TRUE, 3:NA

```

Logistic RIDGE Regression

```

predictors <- label_encoded_clinical
predictors$vital_status <- NULL

# Try multiple K-fold settings for cross-validation
k_values <- c(5, 10)

# Check for NA values in predictors and prints them and Check for NA values in clinical_data$vital_status
if (any(is.na(predictors))) {
  na_rows <- which(rowSums(is.na(predictors)) > 0)
  cat("Rows with NA in predictors:", na_rows, "\n")
}
if (any(is.na(clinical_data$vital_status))) {
  na_rows_vital <- which(is.na(clinical_data$vital_status))
  cat("Rows with NA in vital_status:", na_rows_vital, "\n")
}

## Rows with NA in vital_status: 1010

# Remove rows with NA in vital_status
model_data <- na.omit(data.frame(predictors, vital_status = clinical_data$vital_status))

# Prepare predictor matrix X (numeric matrix) and response y (factor) for glmnet

```



```

X <- as.matrix(model_data[, setdiff(colnames(model_data), "vital_status"), drop = FALSE])
y <- factor(model_data$vital_status)

# Full ridge path fit (used for coefficient shrinkage plot)
ridge_path_fit <- glmnet(X, y, family = "binomial", alpha = 0, intercept = FALSE)

# Fit logistic ridge models for each K value
ridge_runs <- lapply(k_values, function(k) {
  fit <- cv.glmnet(X, y, family = "binomial", alpha = 0, nfolds = k, intercept = FALSE)
  lambda_min <- fit$lambda.min
  lambda_1se <- fit$lambda.1se
  pred_prob <- predict(fit, newx = X, s = "lambda.min", type = "response")
  threshold <- 0.5
  pred_class <- ifelse(pred_prob > threshold, levels(y)[2], levels(y)[1])
  conf_mat <- table(Predicted = pred_class, Observed = y)
  accuracy <- sum(diag(conf_mat)) / sum(conf_mat)
  list(k = k,
       fit = fit,
       lambda_min = lambda_min,
       lambda_1se = lambda_1se,
       pred_prob = pred_prob,
       pred_class = pred_class,
       conf_mat = conf_mat,
       accuracy = accuracy)
})

# Summarize CV outcomes
ridge_summary <- data.frame(
  K = sapply(ridge_runs, `[`, "k"),
  LambdaMin = signif(sapply(ridge_runs, `[`, "lambda_min"), 3),
  Lambda1se = signif(sapply(ridge_runs, `[`, "lambda_1se"), 3),
  Accuracy = round(sapply(ridge_runs, `[`, "accuracy"), 3)
)
print(ridge_summary)

##      K LambdaMin Lambda1se Accuracy
## 1  5      0.334      0.531      0.847
## 2 10      0.334      0.531      0.847

# Select the run with the best accuracy for visualization
best_idx <- which.max(sapply(ridge_runs, `[`, "accuracy"))
best_run <- ridge_runs[[best_idx]]
cat(sprintf("\nBest run: K = %d (accuracy = %.3f, lambda.min = %.4g, lambda.1se = %.4g)\n",
          best_run$k, best_run$accuracy, best_run$lambda_min, best_run$lambda_1se))

##
## Best run: K = 5 (accuracy = 0.847, lambda.min = 0.3335, lambda.1se = 0.531)

ridge_conf_mat_matrix <- as.matrix(best_run$conf_mat)
image(t(ridge_conf_mat_matrix[nrow(ridge_conf_mat_matrix):1, ]),
      axes = FALSE,
      col = colorRampPalette(c("#f7fbff", "#6baed6", "#08306b"))(64),
      xlab = "Observed",
      ylab = "Predicted",
      main = "Ridge Confusion Matrix")

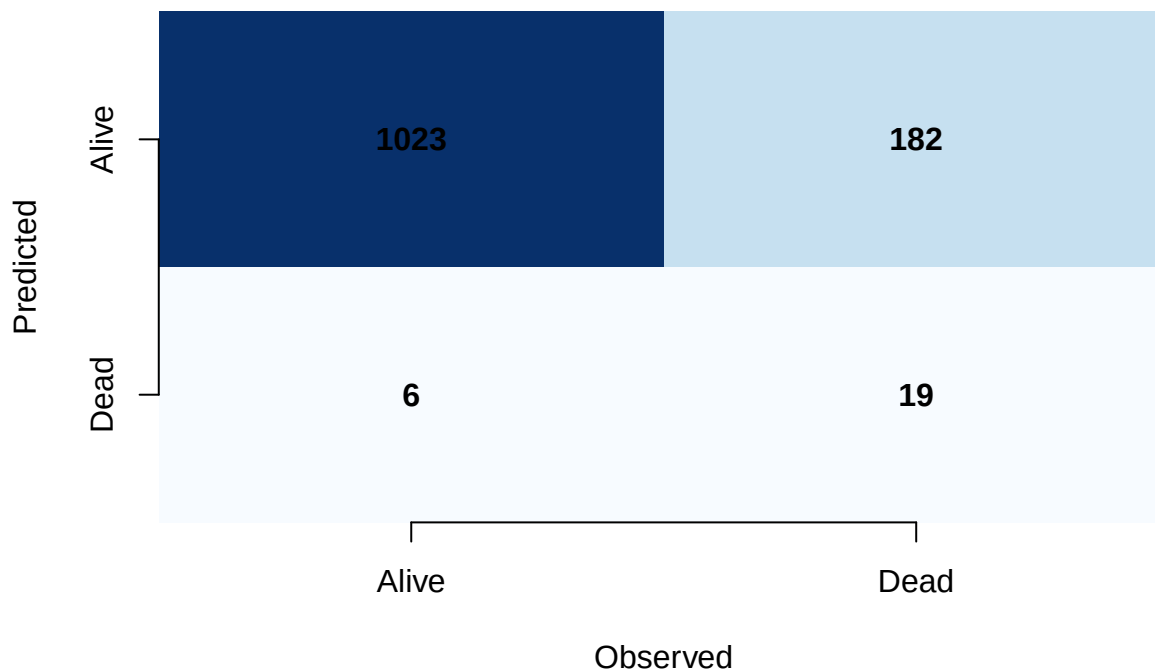
```

```

axis(1,
     at = seq(0, 1, length.out = ncol(ridge_conf_mat_matrix)),
     labels = colnames(ridge_conf_mat_matrix))
axis(2,
     at = seq(0, 1, length.out = nrow(ridge_conf_mat_matrix)),
     labels = rev(rownames(ridge_conf_mat_matrix)))
text(rep(seq(0, 1, length.out = ncol(ridge_conf_mat_matrix)), each = nrow(ridge_conf_mat_matrix)),
     rep(seq(0, 1, length.out = nrow(ridge_conf_mat_matrix)), times = ncol(ridge_conf_mat_matrix)),
     labels = as.vector(ridge_conf_mat_matrix[rev(seq_len(nrow(ridge_conf_mat_matrix))), ]),
     col = "black", font = 2)

```

Ridge Confusion Matrix



```

# Confusion matrices for lambda.min and lambda.1se using the best ridge run
ridge_prob_min <- as.numeric(predict(best_run$fit, newx = X, s = "lambda.min", type = "response"))
ridge_class_min <- ifelse(ridge_prob_min > 0.5, levels(y)[2], levels(y)[1])
ridge_cm_min <- table(Predicted = ridge_class_min, Observed = y)
ridge_prob_1se <- as.numeric(predict(best_run$fit, newx = X, s = "lambda.1se", type = "response"))
ridge_class_1se <- ifelse(ridge_prob_1se > 0.5, levels(y)[2], levels(y)[1])
ridge_cm_1se <- table(Predicted = ridge_class_1se, Observed = y)
# Compute accuracies for display on the confusion matrix plots
ridge_prob_min_plot <- as.numeric(predict(best_run$fit, newx = X, s = "lambda.min", type = "response"))
ridge_class_min_plot <- ifelse(ridge_prob_min_plot > 0.5, levels(y)[2], levels(y)[1])
ridge_acc_min_plot <- mean(ridge_class_min_plot == y)
ridge_prob_1se_plot <- as.numeric(predict(best_run$fit, newx = X, s = "lambda.1se", type = "response"))
ridge_class_1se_plot <- ifelse(ridge_prob_1se_plot > 0.5, levels(y)[2], levels(y)[1])
ridge_acc_1se_plot <- mean(ridge_class_1se_plot == y)

plot_cm_ridge <- function(cm, title, palette = c("#f7fbff", "#6baed6", "#08306b")) {
  m <- as.matrix(cm)
  image(t(m[nrow(m):1, , drop = FALSE]), axes = FALSE,
        col = colorRampPalette(palette)(64), main = title)
}

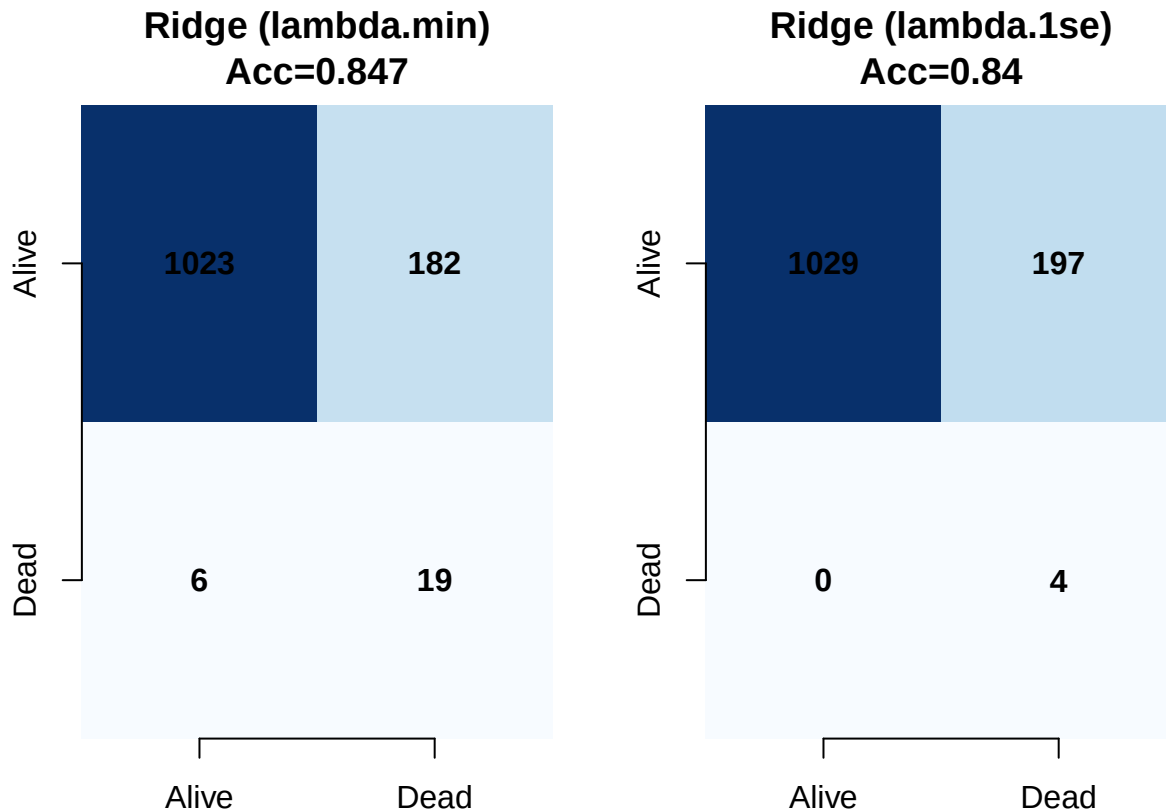
```

```

axis(1, at = seq(0, 1, length.out = ncol(m)), labels = colnames(m))
axis(2, at = seq(0, 1, length.out = nrow(m)), labels = rev(rownames(m)))
text(rep(seq(0, 1, length.out = ncol(m)), each = nrow(m)),
     rep(seq(0, 1, length.out = nrow(m)), times = ncol(m)),
     labels = as.vector(m[rev(seq_len(nrow(m))), ]), col = "black", font = 2)
}

par(mfrow = c(1, 2), mar = c(3, 3, 3, 1))
plot_cm_ridge(ridge_cm_min, paste0("Ridge (lambda.min)\nAcc=", round(ridge_acc_min_plot, 3)))
plot_cm_ridge(ridge_cm_1se, paste0("Ridge (lambda.1se)\nAcc=", round(ridge_acc_1se_plot, 3)))

```



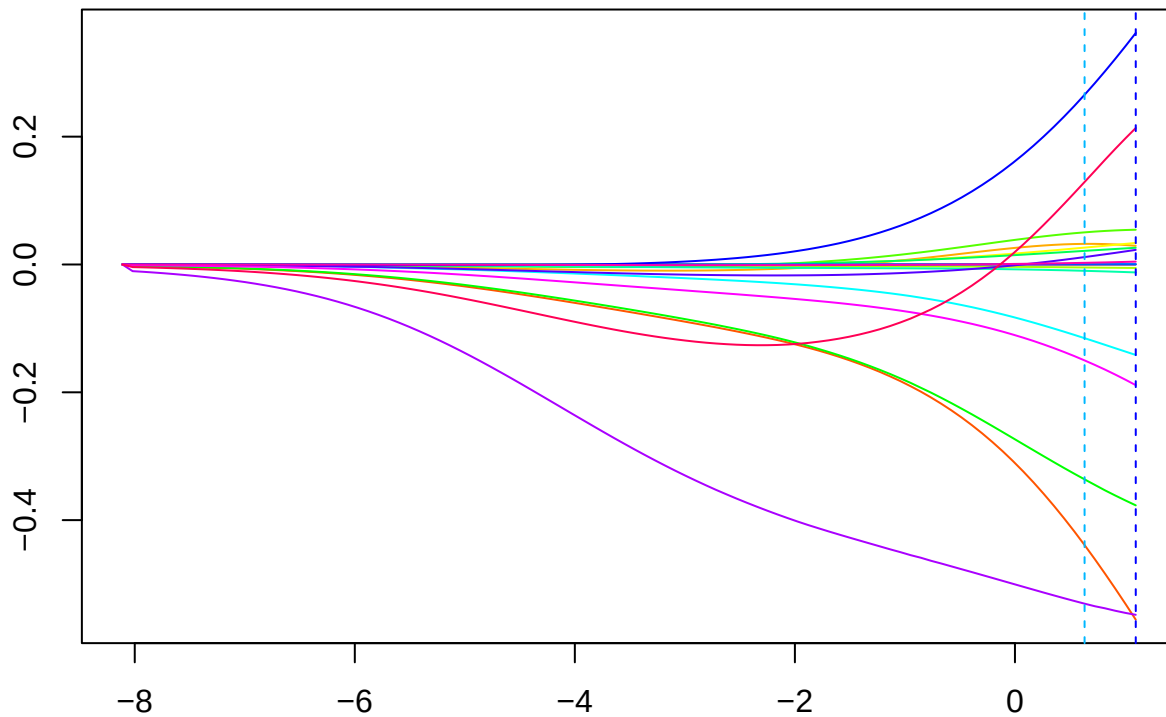
```

par(mfrow = c(1, 1))

# Ridge coefficient paths
lambda_log_vals <- -log(ridge_path_fit$lambda)
matplot(
  lambda_log_vals,
  t(as.matrix(ridge_path_fit$beta)),
  type = "l",
  lty = 1,
  col = rainbow(ncol(X)),
  xlab = "-log(lambda)",
  ylab = "Coefficients",
  main = "Ridge Coefficient Paths (alpha = 0)"
)
abline(v = -log(best_run$lambda_min), col = "blue", lty = 2)
abline(v = -log(best_run$lambda_1se), col = "#00b7ff", lty = 2)

```

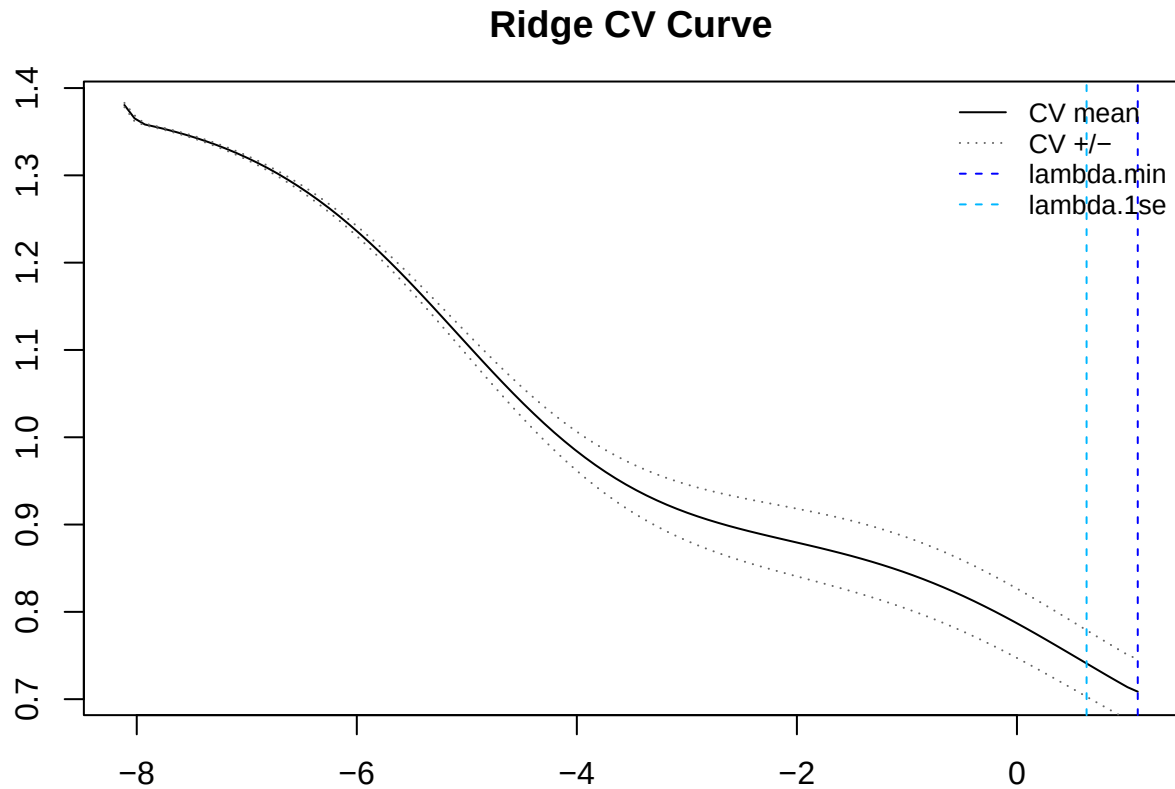
Ridge Coefficient Paths (alpha = 0)



```
# CV curve for ridge
ridge_lambda_log <- -log(best_run$fit$lambda)
plot(ridge_lambda_log, best_run$fit$cvm, type = "l",
     xlab = "-log(lambda)", ylab = "Mean CV deviance",
     main = sprintf("Ridge CV Curve", best_run$k))
```

```
## Warning in sprintf("Ridge CV Curve", best_run$k): one argument not used by
## format 'Ridge CV Curve'
```

```
lines(ridge_lambda_log, best_run$fit$cvup, lty = 3, col = "gray40")
lines(ridge_lambda_log, best_run$fit$cvlo, lty = 3, col = "gray40")
abline(v = -log(best_run$lambda_min), col = "blue", lty = 2)
abline(v = -log(best_run$lambda_1se), col = "#00b7ff", lty = 2)
legend("topright", bty = "n", cex = 0.8,
     legend = c("CV mean", "CV +/-", "lambda.min", "lambda.1se"),
     lty = c(1, 3, 2, 2), col = c("black", "gray40", "blue", "#00b7ff"))
```



Ridge accuracy (lambda.min vs lambda.1se)

```
acc_threshold <- 0.5

# Accuracy at lambda.min
ridge_prob_min <- as.numeric(predict(best_run$fit, newx = X, s = "lambda.min", type = "response"))
ridge_class_min <- ifelse(ridge_prob_min > acc_threshold, levels(y)[2], levels(y)[1])
ridge_acc_min <- mean(ridge_class_min == y)

# Accuracy at lambda.1se
ridge_prob_1se <- as.numeric(predict(best_run$fit, newx = X, s = "lambda.1se", type = "response"))
ridge_class_1se <- ifelse(ridge_prob_1se > acc_threshold, levels(y)[2], levels(y)[1])
ridge_acc_1se <- mean(ridge_class_1se == y)

data.frame(
  Lambda = c("lambda.min", "lambda.1se"),
  Accuracy = round(c(ridge_acc_min, ridge_acc_1se), 3)
)

##      Lambda Accuracy
## 1 lambda.min    0.847
## 2 lambda.1se    0.840
```

Most important variables

Calcul des variables les plus importantes pour le minimum :

```
coef_ridge_min <- coef(best_run$fit, s = "lambda.min")
coef_ridge_min_vec <- as.numeric(coef_ridge_min)
names(coef_ridge_min_vec) <- rownames(coef_ridge_min)
important_vars_ridge_min <- sort(abs(coef_ridge_min_vec), decreasing = TRUE)
head(important_vars_ridge_min, 5)
```

```
##                tissue_type                gender
##                0.5557430                0.5481611
## diagnosis_is_primary_disease follow_ups_disease_response
##                0.3768788                0.3621960
##                age_is_obfuscated
##                0.2135292
```

Détermination des variables les plus importantes pour le 1 standard error :

```
coef_ridge_1se <- coef(best_run$fit, s = "lambda.1se")
coef_ridge_1se_vec <- as.numeric(coef_ridge_1se)
names(coef_ridge_1se_vec) <- rownames(coef_ridge_1se)
important_vars_ridge_1se <- sort(abs(coef_ridge_1se_vec), decreasing = TRUE)
head(important_vars_ridge_1se, 5)
```

```
##                gender                tissue_type
##                0.5307017                0.4390308
## diagnosis_is_primary_disease follow_ups_disease_response
##                0.3362342                0.2655108
##                ethnicity
##                0.1500845
```

Clinical data with Genes (Elastic Net)

We now combine the label-encoded clinical predictors with the gene expressions and fit a logistic Elastic Net (mixing L1/L2). Alpha is chosen by cross-validation over a small grid. We reuse the same plotting templates as above: confusion matrices at `lambda.min/lambda.1se`, coefficient paths, and a custom CV curve without top labels.

Data prep: align outcome, drop rows with any missing values across clinical and gene features, and build the combined predictor matrix.

```
y_en <- factor(clinical_data$vital_status)
keep_en <- complete.cases(y_en, GeneX)
if (any(!keep_en)) cat("Removing", sum(!keep_en), "rows with missing values before elastic net\n")
```

```
## Removing 1 rows with missing values before elastic net
```

```
y_en <- y_en[keep_en]

pos_label_en <- if ("Dead" %in% levels(y_en)) "Dead" else levels(y_en)[1]
neg_label_en <- setdiff(levels(y_en), pos_label_en)
if (length(neg_label_en) == 0) neg_label_en <- pos_label_en
y_en_bin <- as.numeric(y_en == pos_label_en)

X_genes_en <- as.matrix(GeneX[keep_en, , drop = FALSE])
```

```
X_clin_en <- as.matrix(label_encoded_clinical[keep_en, setdiff(colnames(label_encoded_clinical), "vital")]
X_en <- cbind(X_clin_en, X_genes_en)
```

Select alpha by cross-validation on a small grid, then refit the path at the best alpha.

```
alpha_grid <- c(0, 0.25, 0.5, 0.75, 1)
cv_list <- lapply(alpha_grid, function(a) {
  cv.glmnet(X_en, y_en_bin, alpha = a, family = "binomial", grouped = FALSE, nfolds = 10)
})
cv_means <- sapply(cv_list, function(cv) min(cv$cvm))
best_alpha <- alpha_grid[which.min(cv_means)]
cv_en <- cv_list[[which.min(cv_means)]]

elastic_path_fit <- glmnet(X_en, y_en_bin, alpha = best_alpha, family = "binomial")

cat("Selected alpha (EN):", best_alpha, "\n")
```

```
## Selected alpha (EN): 0.25
```

```
cat("lambda.min (EN):", cv_en$lambda.min, "\n")
```

```
## lambda.min (EN): 0.03562094
```

```
cat("lambda.1se (EN):", cv_en$lambda.1se, "\n")
```

```
## lambda.1se (EN): 0.08228898
```

Compute predictions, confusion matrices, and accuracies at lambda.min and lambda.1se.

```
p_en_min <- as.numeric(predict(cv_en, newx = X_en, s = "lambda.min", type = "response"))
p_en_1se <- as.numeric(predict(cv_en, newx = X_en, s = "lambda.1se", type = "response"))

obs_label_en <- ifelse(y_en_bin == 1, pos_label_en, neg_label_en)
pred_label_en_min <- ifelse(p_en_min > 0.5, pos_label_en, neg_label_en)
pred_label_en_1se <- ifelse(p_en_1se > 0.5, pos_label_en, neg_label_en)
label_order_en <- unique(c(neg_label_en, pos_label_en))

cm_en_min <- table(
  Predicted = factor(pred_label_en_min, levels = label_order_en),
  Observed = factor(obs_label_en, levels = label_order_en)
)
cm_en_1se <- table(
  Predicted = factor(pred_label_en_1se, levels = label_order_en),
  Observed = factor(obs_label_en, levels = label_order_en)
)

acc_en_min <- sum(diag(cm_en_min)) / sum(cm_en_min)
acc_en_1se <- sum(diag(cm_en_1se)) / sum(cm_en_1se)

cat(sprintf("Elastic Net Acc (lambda.min): %.3f\n", acc_en_min))
```

```
## Elastic Net Acc (lambda.min): 0.936
```

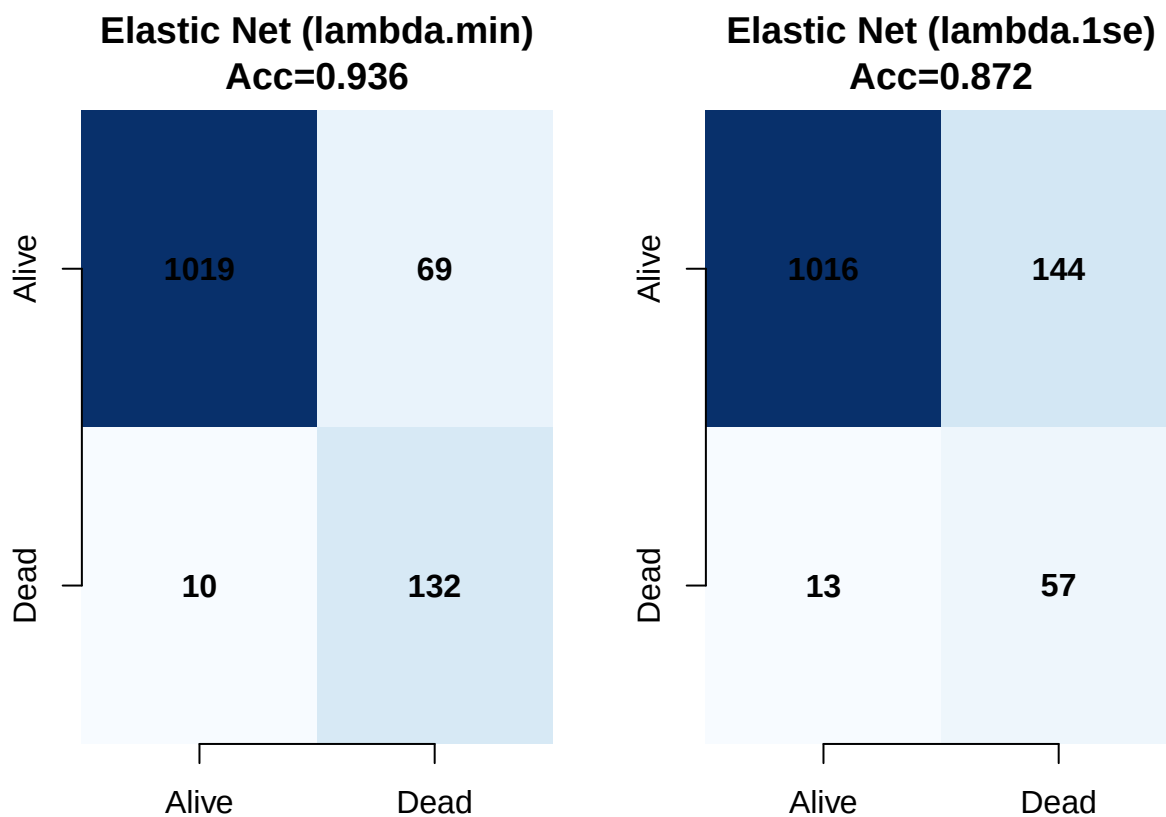
```
cat(sprintf("Elastic Net Acc (lambda.1se): %.3f\n", acc_en_1se))
```

```
## Elastic Net Acc (lambda.1se): 0.872
```

Plot confusion matrices for lambda.min and lambda.1se.

```
plot_cm_en <- function(cm, title, palette = c("#f7fbff", "#6baed6", "#08306b")) {
  m <- as.matrix(cm)
  image(t(m[nrow(m):1, , drop = FALSE])), axes = FALSE,
    col = colorRampPalette(palette)(64), main = title)
  axis(1, at = seq(0, 1, length.out = ncol(m)), labels = colnames(m))
  axis(2, at = seq(0, 1, length.out = nrow(m)), labels = rev(rownames(m)))
  text(rep(seq(0, 1, length.out = ncol(m)), each = nrow(m)),
    rep(seq(0, 1, length.out = nrow(m)), times = ncol(m)),
    labels = as.vector(m[rev(seq_len(nrow(m))), ]), col = "black", font = 2)
}

par(mfrow = c(1, 2), mar = c(3, 3, 3, 1))
plot_cm_en(cm_en_min, paste0("Elastic Net (lambda.min)\nAcc=", round(acc_en_min, 3)))
plot_cm_en(cm_en_1se, paste0("Elastic Net (lambda.1se)\nAcc=", round(acc_en_1se, 3)))
```



```
par(mfrow = c(1, 1))
```

Plot coefficient paths for lambda.min and lambda.1se, and the CV curve (-log lambda) with the chosen alpha highlighted.

```
# Coefficient paths at lambda.min
coef_en_min <- coef(elastic_path_fit, s = cv_en$lambda.min)
coef_en_min_vec <- as.numeric(coef_en_min)
names(coef_en_min_vec) <- rownames(coef_en_min)
important_vars_en_min <- sort(abs(coef_en_min_vec), decreasing = TRUE)
cat("Top coefficients (lambda.min):\n")
```

```
## Top coefficients (lambda.min):
```



```

print(head(important_vars_en_min, 5))

##              (Intercept) follow_ups_disease_response
##              3.7861819          0.7819526
## diagnosis_is_primary_disease          ethnicity
##              0.3364486          0.2447340
##      classification_of_tumor
##              0.1813915

# Coefficient paths at lambda.1se
coef_en_1se <- coef(elastic_path_fit, s = cv_en$lambda.1se)
coef_en_1se_vec <- as.numeric(coef_en_1se)
names(coef_en_1se_vec) <- rownames(coef_en_1se)
important_vars_en_1se <- sort(abs(coef_en_1se_vec), decreasing = TRUE)
cat("Top coefficients (lambda.1se):\n")

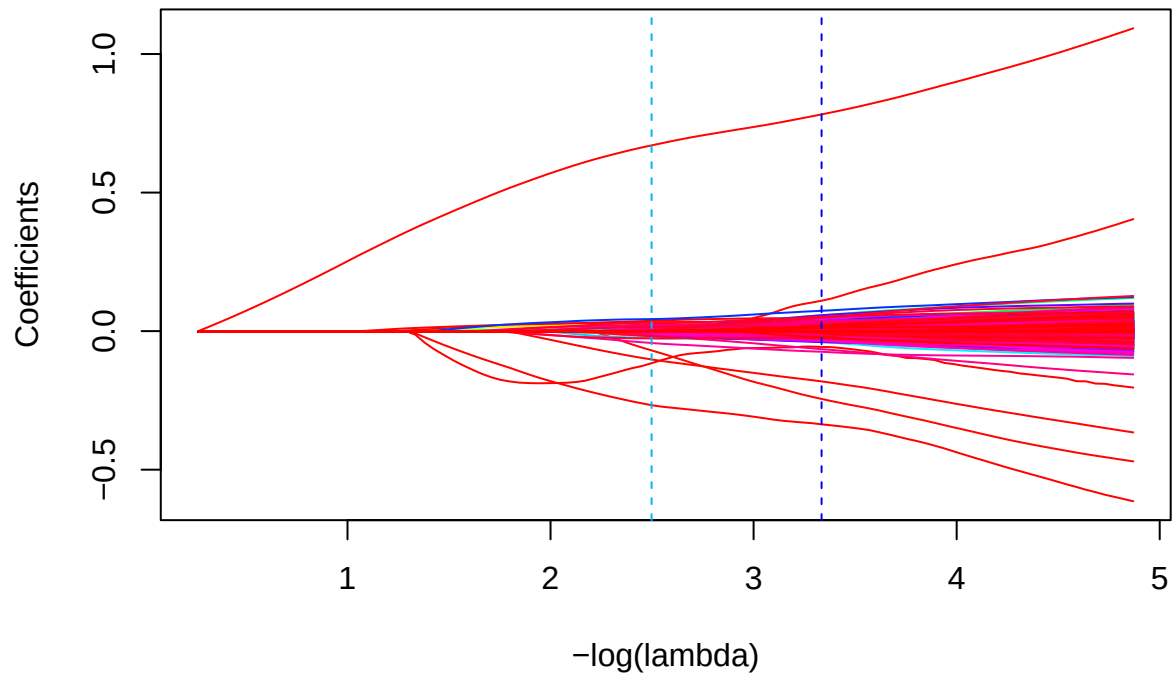
## Top coefficients (lambda.1se):
print(head(important_vars_en_1se, 5))

##              (Intercept) follow_ups_disease_response
##              2.9726049          0.6703086
## diagnosis_is_primary_disease          tissue_type
##              0.2672465          0.1156406
##      classification_of_tumor
##              0.1021381

# Coefficient path plot
matplot(
  -log(elastic_path_fit$lambda),
  t(as.matrix(elastic_path_fit$beta)),
  type = "l", lty = 1, col = rainbow(ncol(X_en)),
  xlab = "-log(lambda)", ylab = "Coefficients",
  main = sprintf("Elastic Net Coefficient Paths (alpha = %.2f)", best_alpha)
)
abline(v = -log(cv_en$lambda.min), col = "blue", lty = 2)
abline(v = -log(cv_en$lambda.1se), col = "#00b7ff", lty = 2)

```

Elastic Net Coefficient Paths (alpha = 0.25)



```
# CV curve
lambda_en_nlog <- -log(cv_en$lambda)
plot(lambda_en_nlog, cv_en$cvm, type = "l",
      xlab = "-log(lambda)", ylab = "Mean CV deviance",
      main = sprintf("Elastic Net CV Curve (alpha = %.2f)", best_alpha))
lines(lambda_en_nlog, cv_en$cvup, lty = 3, col = "gray40")
lines(lambda_en_nlog, cv_en$cvlo, lty = 3, col = "gray40")
abline(v = -log(cv_en$lambda.min), col = "blue", lty = 2)
abline(v = -log(cv_en$lambda.1se), col = "#00b7ff", lty = 2)
legend("topright", bty = "n", cex = 0.8,
      legend = c("CV mean", "CV +/-", "lambda.min", "lambda.1se"),
      lty = c(1, 3, 2, 2), col = c("black", "gray40", "blue", "#00b7ff"))
```

Elastic Net CV Curve (alpha = 0.25)

